

STAT 8670 - Computational Methods in Statistics

2026 Fall Course Notes

Chi-Kuang Yeh

2026-08-18

Table of contents

Preface	5
Description	5
Prerequisites	5
Instructor	5
Office Hour	5
Grade Distribution	5
Assignment	6
Midterm	6
Project	6
Topics and Corresponding Lectures	6
Recommended Textbooks	6
Side Readings	7
1 Data Structure and R Programming	8
1.1 Data type	8
1.1.1 To change data type	9
1.2 Operators	9
1.2.1 Comparison Operator	9
1.2.2 Logical Operator	10
1.3 Indexing	10
1.4 Naming	10
1.5 Array and Matrix	11
1.6 Key and Value Pair	12
1.7 Data Frame	13
1.8 Apply function	14
1.9 Tidyverse	15
1.10 Pipe	16
1.10.1 Some rules	17
1.11 Questions in class	17
1.11.1 Lecture 1, August 25, 2025	17
1.11.2 Lecture 2, August 27, 2025	17

I	Appendix	20
2	Appendix: Introduction to R	21
2.1	Why R?	21
2.2	The working environment	22
2.2.1	RStudio	22
2.2.2	R scripts and Quarto documents	22
2.2.3	A project-oriented workflow	23
2.3	Learning objectives	23
2.4	R as a computing environment	23
2.5	Expressions, objects, and names	25
2.6	Atomic vectors: the basic unit of computation	26
2.6.1	Vectorization	26
2.6.2	Element-wise operations and recycling	27
2.6.3	Coercion	28
2.7	Missing and special numerical values	29
2.8	Indexing and subsetting	30
2.9	Data structures are vectors plus attributes	31
2.9.1	Matrices	31
2.9.2	Lists	32
2.9.3	Factors	33
2.10	Data frames: R's rectangular data structure	34
2.10.1	Creating a data frame	34
2.10.2	Inspecting a data frame	35
2.10.3	Extracting rows, columns, and elements	36
2.10.4	Adding and modifying variables	38
2.10.5	Importing rectangular data	38
2.11	A first look at the tidyverse	39
2.11.1	Data frames and tibbles	40
2.11.2	Pipes and data verbs	41
2.11.3	Operating across several columns	43
2.11.4	Base R and tidyverse are complementary	43
2.12	Functions and function calls	44
2.12.1	Argument matching	45
2.12.2	Lazy evaluation	46
2.13	Environments and lexical scoping	46
2.14	Memory: copy-on-modify	47
2.15	Iteration: vectorization, loops, and the apply family	49
2.15.1	<code>apply()</code> : rows or columns of a matrix	50
2.15.2	Why <code>apply()</code> can be dangerous for data frames	51
2.15.3	<code>lapply()</code> : apply a function and return a list	52
2.15.4	<code>sapply()</code> and <code>vapply()</code> : simplified output	54
2.15.5	<code>tapply()</code> : grouped operations on a vector	55

2.15.6	<code>Map()</code> and <code>mapply()</code> : parallel iteration	55
2.15.7	Choosing an iteration tool	56
2.16	Numerical computation is finite-precision computation	56
2.17	Randomness and reproducibility	57
2.18	Packages, help, and error messages	59
2.19	A computational workflow for this course	60
2.20	In-class exercises	61
2.20.1	Exercise 8: Diagnose coercion	62
2.21	Takeaways	62
2.22	Optional challenge	63

Preface

Description

Topics included are optimization, numerical integration, bootstrapping, cross-validation and Jackknife, density estimation, smoothing, and use of the statistical computer package of S-plus/R.

Prerequisites

MATH 4752/6752 – Mathematical Statistics II, and the ability to program in a high-level language.

Instructor

[Chi-Kuang Yeh](#), I am an Assistant Professor in the Department of Mathematics and Statistics, Georgia State University.

- Office: Suite 1407, 25 Park Place.
- Email: cych@gsu.edu.

Office Hour

Office hour is TBA.

Grade Distribution

- Assignments: 40%
- Exam: 30%
- Project: 30%

Assignment

- ☐ Assignment 1: TBA

Midterm

- ☐ Midterm: TBA

Project

- ☐ Report: TBA
- ☐ Presentation:

Topics and Corresponding Lectures

Those chapters are based on the lecture notes. This part will be updated frequently.

Status	Topic	Lecture Covered
	R Programming	1–2
	Numerical Approaches and Optimization	TBA
	Probability Distribution	
	Random Variable Generation	
	Monte Carlo and Integration	TBA
	Cross-Validation	TBA
	Bootstrap and Jackknife	TBA
	Selective topics	TBA

Recommended Textbooks

- Givens, G.H. and Hoeting, J.A. (2012). *Computational Statistics*. Wiley, New York.
- Rizzo, M.L. (2007) *Statistical Computing with R*. CRC Press, Boca Raton.
- Hothorn, T. and Everitt, B.S. (2006). *A Handbook of Statistical Analyses Using R*. CRC Press, Boca Raton.

Side Readings

- Wickham, H., Çetinkaya-Rundel, M. and Grolemund, G. (2023). *R for Data Science*. O'Reilly.

1 Data Structure and R Programming

Data types, operators, variables

Two basic types of objects: (1) data & (2) functions

- Data: can be a number, a vector, a matrix, a dataframe, a list or other datatypes
- Function: a function is a set of instructions that takes input, processes it, and returns output. Functions can be built-in or user-defined.

1.1 Data type

- Boolean/Logical: Yes or No, Head or Tail, True or False
- Integers: Whole numbers $\mathbb{Z}$, e.g., 1, 2, 3, -1, -2, -3
- Characters: Text strings, e.g., “Hello”, “World.”
- Floats: Noninteger fractional numbers, e.g., π , e .
- Missing data: NA in R, which stands for “Not Available.” It is used to represent missing or undefined values in a dataset.

```
day <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday")
weather <- c("Raining", "Sunny", NA, "Windy", "Snowing")
data.frame(rbind(day, weather))
```

	X1	X2	X3	X4	X5
day	Monday	Tuesday	Wednesday	Thursday	Friday
weather	Raining	Sunny	<NA>	Windy	Snowing

- Other more complex types

1.1.1 To change data type

You may change the data type using the following functions, but the chance is that some of the information will be missing. Do this with caution!

```
x <- pi  
print(x)
```

```
[1] 3.141593
```

```
x_int <- as.integer(x)  
print(x_int)
```

```
[1] 3
```

Some of the conversion functions:

- `as.integer()`: Convert to integer.
- `as.numeric()`: Convert to numeric (float).
- `as.character()`: Convert to character.
- `as.logical()`: Convert to logical (boolean).
- `as.Date()`: Convert to date.
- `as.factor()`: Convert to factor (categorical variable).
- `as.list()`: Convert to list.
- `as.matrix()`: Convert to matrix.
- `as.data.frame()`: Convert to data frame.
- `as.vector()`: Convert to vector.
- `as.complex()`: Convert to complex number.

1.2 Operators

- Unary: With only **one** argument. E.g., `-x` (negation), `!x` (logical negation).
- Binary: With **two** arguments. E.g., `x + y` (addition), `x - y` (subtraction), `x * y` (multiplication), `x / y` (division).

1.2.1 Comparison Operator

Comparing two objects. E.g., `x == y` (equal), `x != y` (not equal), `x < y` (less than), `x > y` (greater than), `x <= y` (less than or equal to), `x >= y` (greater than or equal to).

1.2.2 Logical Operator

Logical operators are used to combine or manipulate logical values (TRUE or FALSE). E.g., `x & y` (logical AND), `x | y` (logical OR), `!x` (logical NOT).

We shall note that the logical operators in R are vectorized, `x | y` and `x || y` are different. The former is vectorized, while the latter is not.

```
x <- c(TRUE, FALSE, FALSE)
y <- c(TRUE, FALSE, FALSE)
x | y # [1] TRUE FALSE FALSE
x || y # This will return an error
```

1.3 Indexing

Indexing is a way to access or modify specific elements in a data structure. In **R**, indexing can be done using square brackets `[]` for vectors and matrices, or the `$` operator for data frames. Note that the index starts from **0** in **R**, which is different from some other programming languages like Python.

1.4 Naming

In **R**, you can assign names to objects using the `names()` function. This is useful for making your code more readable and for accessing specific elements in a data structure.

A good practice is to use `_` (underscore) to separate words in variable names, e.g., `my_variable`. This makes the code more readable and easier to understand.

```
# Assign names to a vector
temp <- c(20, 30, 27, 31, 45)
names(temp) <- c("Mon", "Tues", "Wed", "Thurs", "Fri")
print(temp)
```

Mon	Tues	Wed	Thurs	Fri
20	30	27	31	45

```
rownames(temp) <- "Day1" # error
```

```
temp_mat <- matrix(c(20, 30, 27, 31, 45), nrow = 1, ncol = 5)
colnames(temp_mat) <- c("Mon", "Tues", "Wed", "Thurs", "Fri")
rownames(temp_mat) <- "Day1" # error
print(temp_mat)
```

```
      Mon Tues Wed Thurs Fri
Day1  20   30  27    31  45
```

1.5 Array and Matrix

One may define an array or a matrix in **R** using the `array()` or `matrix()` functions, respectively. An array is a multi-dimensional data structure, while a matrix is a two-dimensional array.

```
# Create a 1-dimensional array
array_1d <- array(1:10, dim = 10)
array_1d
```

```
[1]  1  2  3  4  5  6  7  8  9 10
```

```
# Create a 2-dimensional array
array_2d <- array(1:12, dim = c(4, 3))
array_2d
```

```
      [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
```

```
# Create a 3-dimensional array
array_3d <- array(1:24, dim = c(4, 3, 2))
array_3d
```

```
, , 1
```

```
      [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
```

```
[3,]    3    7   11
[4,]    4    8   12
```

```
, , 2
```

```
      [,1] [,2] [,3]
[1,]   13   17   21
[2,]   14   18   22
[3,]   15   19   23
[4,]   16   20   24
```

```
# Create a matrix
my_matrix <- matrix(1:12, nrow = 4, ncol = 3)
my_matrix
```

```
      [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
```

Note here, the matrix is a special case of an array, where the number of dimensions is exactly 2.

```
is.matrix(array_2d) # TRUE
is.matrix(my_matrix) # TRUE

is.array(array_2d) # TRUE
is.array(my_matrix) # TRUE
```

1.6 Key and Value Pair

Key-Value Pair is a data structure that consists of a key and its corresponding value. In **R**, this can be implemented using named vectors, lists, or data frames. Usually, the most commonly used case is in the lists and data frames. The values can be extra by providing the corresponding key

```
key1 <- "Tues"
value1 <- 32
key2 <- "Wed"
value2 <- 28

list_temp <- list()
list_temp[[ key1 ]] <- value1
list_temp[[ key2 ]] <- value2

print(list_temp)
```

```
$Tues
[1] 32
```

```
$Wed
[1] 28
```

```
## Now providing a key - Tues
### First way
list_temp[["Tues"]]
```

```
[1] 32
```

```
### Second way
list_temp$Tues
```

```
[1] 32
```

1.7 Data Frame

Dataframe is a two-dimensional, tabular data structure in R that can hold different types of variables (numeric, character, factor, etc.) in each column. It is similar to a spreadsheet or SQL table.

```
iris <- datasets::iris
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

1.8 Apply function

The `apply()` function is the basic model of the family of apply functions in R, which includes specific functions like `lapply()`, `sapply()`, `tapply()`, `mapply()`, `vapply()`, `rapply()`, `bapply()`, `eapply()`, and others. These functions are used to apply a function to elements of a data structure (like a vector, list, or data frame) in a (sometimes) more efficient and concise way than using loops.

```
x <- cbind(x1 = 3, x2 = c(4:1, 2:5))
dimnames(x)[[1]] <- letters[1:8]
print(x)
```

```
  x1 x2
a  3  4
b  3  3
c  3  2
d  3  1
e  3  2
f  3  3
g  3  4
h  3  5
```

```
apply(x, MARGIN = 2, mean) #apply the mean function to their "columns"
```

```
x1 x2
3  3
```

```
col.sums <- apply(x, MARGIN = 2, sum) #apply the sum function to their "columns"
row.sums <- apply(x, MARGIN = 1, sum) #apply the sum function to their "rows"
rbind(cbind(x, Rtot = row.sums), Ctot = c(col.sums, sum(col.sums)))
```

	x1	x2	Rtot
a	3	4	7
b	3	3	6
c	3	2	5
d	3	1	4
e	3	2	5
f	3	3	6
g	3	4	7
h	3	5	8
Ctot	24	24	48

Some of the commonly used apply functions:

- **lapply**: Apply a Function over a List or Vector
- **sapply**: a user-friendly version and wrapper of lapply by default returning a vector, matrix
- **vapply**: similar to sapply, but has a pre-specified type of return value, so it can be safer (and sometimes faster) to use.

1.9 Tidyverse

The tidyverse is a collection of open source packages for the R programming language introduced by Hadley Wickham and his team that “share an underlying design philosophy, grammar, and data structures” of tidy data. Characteristic features of tidyverse packages include extensive use of non-standard evaluation and encouraging piping.

```
## Load all tidyverse packages
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.2
v ggplot2    4.0.0      v tibble     3.3.0
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.1.0

-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
## Or load specific packages in the tidy family
library(dplyr) # Data manipulation
library(ggplot2) # Data visualization
library(readr) # Data import
library(tibble) # Tidy data frames
library(tidyr) # Data tidying
# ...
```

1.10 Pipe

Pipe operator `|>` (native after R version 4.0) or `%>%` (from `magrittr` package) is a powerful tool in **R** that allows you to chain together multiple operations in a clear and concise way. It takes the output of one function and passes it as the first argument to the next function.

For example, we can write

```
set.seed(777)
x <- rnorm(5)

## Without using pipe
print(round(mean(x), 2))
```

```
[1] 0.37
```

```
## Using pipe
x |>
  mean() |> # applying the mean function
  round(2) |> #round to 2nd decimal place
  print()
```

```
[1] 0.37
```

We can see that, without using the pipe, if we are applying multiple functions to the same object, we may have hard time to track. This can make the code less readable and harder to maintain. On the other hand, using pipe, we can clearly see the sequence of operations being applied to the data, making it easier to understand and modify.

1.10.1 Some rules

|> should **always have a space before it** and should typically **be the last thing on a line**. This simplifies adding new steps, reorganizing existing ones, and modifying elements within each step.

Note that all of the packages in the tidyverse family support the pipe operator (except `ggplot2!`), so you can use it with any of them.

1.11 Questions in class

1.11.1 Lecture 1, August 25, 2025

Q1. If I know Python already, why learn R?

Reply: My general take are 1). R is more specialized for statistical analysis and data visualization, while Python is a more general-purpose programming language. 2). R has a rich ecosystem of packages and libraries specifically designed for statistical computing, making it a popular choice among statisticians and data scientists. 3). R's syntax and data structures are often more intuitive for statistical tasks, which can lead to faster development and easier collaboration with other statisticians. 4). Also, the tidyverse ecosystem including *ggplot* and others are a big plus when dealing with big dataframes. 5). They are not meant to replace each other, but work as a complement.

Q2. Why my installation of R sometimes failed on a Windows machine?

Reply: There are many reasons. One of the most common reasons is that you may need to manually add the path to the environment variable.

1.11.2 Lecture 2, August 27, 2025

Q1. What's the difference of using `apply` v.s. `looping` in R?

Reply: The `apply` functions are often faster and more efficient than looping, especially for large datasets, because they have done some vectorization under the hood. Also, it has much higher readability and better conciseness. However, depends on the task, you may want to do the **benchmarking** to see the performance difference.

Q2. How to use `pipe` with two or more variables?

Reply: There are several ways to do this.

1. Within the tidyverse family: One way is to use the `dplyr` package, which provides a set of functions that work well with the pipe operator. For example, you can use the `mutate()` function to create a new variable based on two existing variables. For example, you can do

```
library(dplyr)
library(magrittr) # for %$%
library(purrr)    # for pmap / exec if needed

my_df <- tibble(x = 1:5, y = 6:10)
f <- function(a, b) a + 2*b

my_df %>%
  mutate(z = f(x, y))
```

```
# A tibble: 5 x 3
      x     y     z
  <int> <int> <dbl>
1     1     6    13
2     2     7    16
3     3     8    19
4     4     9    22
5     5    10    25
```

2. Using base R, you may do something like the following through the `magrittr` package's exposition pipe `%$%`:

```
library(magrittr)
# method 1
my_df %$% f(x, y)
```

```
[1] 13 16 19 22 25
```

```
# or use . as a placeholder
# method 2
my_df %>% { f(.$x, .$y) }
```

```
[1] 13 16 19 22 25
```

Some of the materials are adapted from [CMU Stat36-350](#).

A comprehensive reference for all the *tidyverse* tools is [R for Data Science](#).

A comprehensive reference for *ggplot2* is [ggplot2: Elegant Graphics for Data Analysis](#).

Part I

Appendix

2 Appendix: Introduction to R

2.1 Why R?

Modern statistical analyses routinely involve hundreds, thousands, or millions of observations. The calculations are too numerous—and often too complex—to perform reliably by hand. We therefore need software that can manage data, implement statistical methods, automate repeated computations, and document the complete analysis.

R is both a high-level programming language and an environment for statistical computing and graphics. It is especially useful for statistics because it is:

- **free and open source**, so students and researchers can use the same tools;
- **designed for data analysis**, with vectors, missing values, models, and graphics built into the language;
- **extensible**, with a large ecosystem of packages developed by researchers and software developers;
- **reproducible**, because an analysis can be saved as code and rerun from beginning to end;
- **interoperable**, with interfaces to databases and languages such as C++, Python, and Julia.

R is not a menu-driven statistics program. Learning R means learning to express an analysis as a sequence of precise, inspectable computations. That initial investment pays off when an analysis must be revised, repeated, validated, or shared.

i R, RStudio, and Quarto are different tools

R performs the computation. **RStudio** is an integrated development environment that helps us write and run code. **Quarto** turns source documents containing prose, code, and results into HTML, PDF, Word, or presentation output.

2.2 The working environment

2.2.1 RStudio

RStudio is an integrated development environment (IDE) for R, Python, and reproducible publishing. Its default layout contains four panes:

Pane	Main purpose
Source Console	Write and edit scripts and Quarto documents. Send expressions directly to R and view immediate results.
Environment/History	Inspect names in the global environment and review console commands.
Files/Plots/Packages/Help/Viewer	Navigate files, inspect graphics, manage packages, read documentation, and preview output.

Use the console for short experiments. Put any computation that matters in a script or Quarto document so it can be rerun.

2.2.2 R scripts and Quarto documents

An R script is a plain-text file with the extension `.R`. It contains R expressions that can be executed one line, one selection, or one file at a time.

A Quarto document has the extension `.qmd`. It combines:

- a YAML header describing the document;
- prose written in Markdown;
- executable code chunks;
- tables, equations, figures, citations, and cross-references.

For example, an executable R chunk begins with ````\{r}` and ends with `````. Rendering the document starts a computation, runs the enabled chunks in order, and places their results into the output. This makes the narrative and the computation part of the same reproducible record.

2.2.3 A project-oriented workflow

Create one RStudio Project for the course or for each substantial analysis. Use project-relative paths such as `data/survey.csv` rather than hard-coded paths tied to one computer. A simple project might contain:

```
stat8670-project/  
  data/          # original and processed data  
  R/             # reusable functions  
  figures/       # saved figures  
  notes/        # Quarto lecture or analysis files  
  stat8670-project.Rproj
```

Avoid using `setwd()` inside a reproducible document. When a project is opened, its root provides a stable reference point for relative paths.

2.3 Learning objectives

By the end of this lecture, you should be able to:

1. Describe how R reads, evaluates, and prints an expression.
2. Distinguish an object from the name bound to that object.
3. Predict how vectorization, coercion, recycling, and missing values affect a computation.
4. Explain how function calls, environments, lexical scoping, and lazy evaluation work.
5. Describe R's copy-on-modify memory model and its implications for performance.
6. Create, inspect, subset, and transform data frames and tibbles.
7. Choose appropriately among `apply()`, `lapply()`, `vapply()`, grouped operations, and vectorized functions.
8. Use the tidyverse pipe and core `dplyr` verbs to express a data-analysis pipeline.
9. Write small computations that are reproducible, testable, and numerically responsible.

2.4 R as a computing environment

R is both a programming language and an interactive environment for statistical computing. It provides:

- a parser that turns text into expressions;
- an evaluator that computes the value of an expression;
- a workspace containing objects and functions;
- a package system for extending the language;

- graphics, statistical models, simulation tools, and interfaces to compiled code.

At the console, R follows a **read–evaluate–print loop** (REPL):

1. **Read:** parse the text entered by the user.
2. **Evaluate:** compute the value of the expression in the current environment.
3. **Print:** display the result when it is visible.
4. **Loop:** wait for the next expression.

```
2 + 3 * 4
```

```
[1] 14
```

R respects operator precedence, so multiplication is performed before addition. Parentheses make the intended order explicit.

```
(2 + 3) * 4
```

```
[1] 20
```

Assignment usually returns its value invisibly, so the first line below does not print a result. Typing the name on the next line retrieves and prints the value.

```
x <- 2 + 3 * 4  
x
```

```
[1] 14
```

💡 A useful mental model

Most R code can be understood as **function calls that produce objects**. Even an operator such as `+` is a function.

```
`+`(2, 3)
```

```
[1] 5
```

2.5 Expressions, objects, and names

R computes with **objects**. Numbers, character strings, vectors, data frames, fitted models, and functions are all objects. A **name** is a symbol bound to an object in an environment.

```
sample_size <- 100
another_name <- sample_size

sample_size
```

```
[1] 100
```

```
another_name
```

```
[1] 100
```

The assignment operator `<-` creates or changes a binding. It is read as “gets.” Although `=` can also assign in many contexts, using `<-` for ordinary assignment makes code easier to read because `=` is also used to match function arguments.

Names are case-sensitive:

```
n <- 10
N <- 25
c(n = n, N = N)
```

```
  n  N
10 25
```

Use `ls()` to list names in the current environment and `rm()` to remove a binding.

```
temporary_result <- 42
"temporary_result" %in% ls()
```

```
[1] TRUE
```

```
rm(temporary_result)
"temporary_result" %in% ls()
```

```
[1] FALSE
```

Removing a name does not necessarily destroy an object immediately. R reclaims unreachable memory through **garbage collection**.

2.6 Atomic vectors: the basic unit of computation

The fundamental data structure in R is the **vector**. An atomic vector contains zero or more elements of one type. Common atomic types include:

Type	Example	Typical use
logical	TRUE, FALSE	conditions and indicators
integer	2L	counts and indices
double	2, 3.14	numerical computation
character	"R"	text and labels
complex	1 + 2i	complex arithmetic
raw	charToRaw("R")	bytes

```
logical_vector <- c(TRUE, FALSE, TRUE)
integer_vector <- c(1L, 2L, 3L)
double_vector <- c(1, 2, 3)
character_vector <- c("one", "two", "three")
```

```
typeof(logical_vector)
```

```
[1] "logical"
```

```
typeof(integer_vector)
```

```
[1] "integer"
```

```
typeof(double_vector)
```

```
[1] "double"
```

```
typeof(character_vector)
```

```
[1] "character"
```

2.6.1 Vectorization

Many R functions and operators act on an entire vector. This is called **vectorized computation**.

```
x <- c(1, 2, 3, 4)
x^2
```

```
[1] 1 4 9 16
```

```
sqrt(x)
```

```
[1] 1.000000 1.414214 1.732051 2.000000
```

```
sum(x)
```

```
[1] 10
```

```
mean(x)
```

```
[1] 2.5
```

Vectorized code often expresses the statistical operation more directly than element-by-element code. It may also be faster because much of the work is implemented in compiled code.

2.6.2 Element-wise operations and recycling

Arithmetic between vectors is usually element-wise.

```
x <- c(10, 20, 30)
y <- c(1, 2, 3)
x + y
```

```
[1] 11 22 33
```

If one vector is shorter, R repeats—or **recycles**—its values.

```
x + 5
```

```
[1] 15 25 35
```

```
c(10, 20, 30, 40) + c(1, 2)
```

```
[1] 11 22 31 42
```

Recycling is useful when the shorter length divides the longer length exactly. Otherwise, R warns because the result may reflect a mistake.

```
c(10, 20, 30, 40) + c(1, 2, 3)
```

```
Warning in c(10, 20, 30, 40) + c(1, 2, 3): longer object length is not a  
multiple of shorter object length
```

```
[1] 11 22 33 41
```

2.6.3 Coercion

An atomic vector must have one type. When incompatible types are combined, R finds a common type and **coerces** the values.

```
mixed_numeric <- c(TRUE, 2L, 3.5)  
mixed_character <- c(TRUE, 2L, 3.5, "four")  
  
mixed_numeric
```

```
[1] 1.0 2.0 3.5
```

```
typeof(mixed_numeric)
```

```
[1] "double"
```

```
mixed_character
```

```
[1] "TRUE" "2"    "3.5"  "four"
```

```
typeof(mixed_character)
```

```
[1] "character"
```

A simplified coercion hierarchy is:

logical → integer → double → complex → character

Implicit coercion is convenient, but accidental coercion can silently change a computation. Inspect unfamiliar objects with `typeof()`, `class()`, `length()`, and `str()`.

2.7 Missing and special numerical values

R uses `NA` for a missing value. Because an unknown value usually makes a calculation unknown, `NA` propagates.

```
scores <- c(88, NA, 91, 84)
mean(scores)
```

```
[1] NA
```

```
mean(scores, na.rm = TRUE)
```

```
[1] 87.66667
```

```
is.na(scores)
```

```
[1] FALSE TRUE FALSE FALSE
```

Do not test missingness with `x == NA`; use `is.na(x)`.

Other special numerical values include positive and negative infinity and “not a number.”

```
c(1 / 0, -1 / 0, 0 / 0)
```

```
[1] Inf -Inf NaN
```

```
is.finite(c(1, Inf, NA, NaN))
```

```
[1] TRUE FALSE FALSE FALSE
```

`NaN` represents an undefined numerical result and is also treated as missing by `is.na()`.

Three-valued logic matters when conditions contain missing values:

```
c(TRUE & NA, FALSE & NA, TRUE | NA, FALSE | NA)
```

```
[1]    NA FALSE  TRUE    NA
```

2.8 Indexing and subsetting

R offers several ways to select elements. For a vector `x`:

- positive integers select positions;
- negative integers exclude positions;
- logical vectors retain positions that are `TRUE`;
- character vectors select named elements.

```
x <- c(first = 10, second = 20, third = 30, fourth = 40)
```

```
x[c(1, 3)]
```

```
first third
  10    30
```

```
x[-2]
```

```
first  third fourth
   10    30    40
```

```
x[x >= 30]
```

```
third fourth
   30    40
```

```
x[c("second", "fourth")]
```

```
second fourth
   20    40
```

Indices begin at 1, not 0. Mixing positive and negative indices in the same subscript is an error.

 A common surprise

`1:length(x)` behaves badly when `x` has length zero. Prefer `seq_along(x)` when iterating over positions and `seq_len(n)` when iterating to a known length.

```
empty <- numeric(0)
1:length(empty)
```

```
[1] 1 0
```

```
seq_along(empty)
```

```
integer(0)
```

2.9 Data structures are vectors plus attributes

Many R data structures are constructed by attaching attributes to vectors.

2.9.1 Matrices

A matrix is an atomic vector with a `dim` attribute. Its elements must have one common type.

```
A <- matrix(1:6, nrow = 2, ncol = 3)
A
```

```
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```
attributes(A)
```

```
$dim
[1] 2 3
```

```
A[2, 3]
```

```
[1] 6
```

Matrix multiplication uses `%%`; `*` performs element-wise multiplication.

```
B <- matrix(c(1, 0, 0, 1, 1, 1), nrow = 3)
A %% B
```

```
      [,1] [,2]
[1,]     1     9
[2,]     2    12
```

2.9.2 Lists

A list is a vector whose elements may be objects of different types and lengths.

```
fit_summary <- list(
  method = "least squares",
  coefficients = c(intercept = 1.2, slope = 0.8),
  converged = TRUE
)

str(fit_summary)
```

```
List of 3
 $ method      : chr "least squares"
 $ coefficients: Named num [1:2] 1.2 0.8
 ..- attr(*, "names")= chr [1:2] "intercept" "slope"
 $ converged    : logi TRUE
```

```
fit_summary[["coefficients"]]
```

```
intercept    slope
      1.2      0.8
```

```
fit_summary$coefficients
```

```
intercept    slope
      1.2      0.8
```

For a list, `[` returns a sublist, whereas `[[` extracts a single element.

```
fit_summary["coefficients"]
```

```
$coefficients
intercept      slope
          1.2      0.8
```

```
fit_summary[["coefficients"]]
```

```
intercept      slope
          1.2      0.8
```

2.9.3 Factors

A factor represents categorical data using integer codes plus a `levels` attribute.

```
group <- factor(c("control", "treatment", "control"))
group
```

```
[1] control  treatment control
Levels: control treatment
```

```
unclass(group)
```

```
[1] 1 2 1
attr(,"levels")
[1] "control" "treatment"
```

```
levels(group)
```

```
[1] "control" "treatment"
```

The order of factor levels can affect model coefficients and printed summaries, so set levels deliberately when their order matters.

2.10 Data frames: R's rectangular data structure

A **data frame** stores a rectangular data set. Conceptually, it is a named list of equal-length vectors:

- each **row** represents an observational unit;
- each **column** represents a variable;
- every column has the same number of observations;
- different columns may have different types.

This last property distinguishes a data frame from a matrix. A matrix is one atomic vector with dimensions, so every entry must have the same type. A data frame is a list of columns, so numerical, character, logical, factor, and date variables can coexist.

2.10.1 Creating a data frame

```
students <- data.frame(  
  id = 1:6,  
  name = c("Alice", "Bob", "Chen", "Divya", "Elena", "Farah"),  
  program = c("MS", "PhD", "MS", "PhD", "MS", "PhD"),  
  score = c(90, 85, NA, 94, 88, 91),  
  completed = c(TRUE, TRUE, FALSE, TRUE, TRUE, TRUE)  
)  
  
students
```

	id	name	program	score	completed
1	1	Alice	MS	90	TRUE
2	2	Bob	PhD	85	TRUE
3	3	Chen	MS	NA	FALSE
4	4	Divya	PhD	94	TRUE
5	5	Elena	MS	88	TRUE
6	6	Farah	PhD	91	TRUE

Each expression passed to `data.frame()` becomes a column. R recycles a length-one value, but other columns should normally have matching lengths.

2.10.2 Inspecting a data frame

Never begin an analysis by assuming that an imported object has the expected structure. Inspect it.

```
class(students)
```

```
[1] "data.frame"
```

```
dim(students)      # number of rows and columns
```

```
[1] 6 5
```

```
nrow(students)
```

```
[1] 6
```

```
ncol(students)
```

```
[1] 5
```

```
names(students)    # column names
```

```
[1] "id"      "name"    "program" "score"   "completed"
```

```
str(students)      # compact structural summary
```

```
'data.frame':  6 obs. of  5 variables:
 $ id      : int  1 2 3 4 5 6
 $ name    : chr  "Alice" "Bob" "Chen" "Divya" ...
 $ program : chr  "MS" "PhD" "MS" "PhD" ...
 $ score   : num  90 85 NA 94 88 91
 $ completed: logi  TRUE TRUE FALSE TRUE TRUE TRUE
```

```
summary(students)  # variable-by-variable summary
```

	id		name		program		score		completed
Min.	:1.00	Length	:6	Length	:6	Min.	:85.0	Mode	:logical
1st Qu.	:2.25	N.unique	:6	N.unique	:2	1st Qu.	:88.0	FALSE	:1
Median	:3.50	N.blank	:0	N.blank	:0	Median	:90.0	TRUE	:5
Mean	:3.50	Min.nchar	:3	Min.nchar	:2	Mean	:89.6		
3rd Qu.	:4.75	Max.nchar	:5	Max.nchar	:3	3rd Qu.	:91.0		
Max.	:6.00					Max.	:94.0		
						NAs	:1		

```
head(students, 3)
```

```

  id name program score completed
1  1 Alice      MS    90       TRUE
2  2  Bob      PhD    85       TRUE
3  3  Chen      MS    NA      FALSE

```

`str()` is especially valuable: it shows the object class, dimensions, column names, column types, and representative values in one compact display.

2.10.3 Extracting rows, columns, and elements

The expression `students[rows, columns]` uses two-dimensional indexing.

```
students[1, ] # first row, all columns
```

```

  id name program score completed
1  1 Alice      MS    90       TRUE

```

```
students[, c("name", "score")] # all rows, selected columns
```

```

  name score
1 Alice    90
2  Bob     85
3  Chen    NA
4 Divya    94
5 Elena    88
6 Farah    91

```

```
students[students$program == "PhD", ]
```

```
  id name program score completed
2  2  Bob    PhD    85         TRUE
4  4 Divya    PhD    94         TRUE
6  6 Farah    PhD    91         TRUE
```

```
students[1:3, c("name", "score")]
```

```
  name score
1 Alice    90
2  Bob     85
3  Chen    NA
```

```
students[2, "score"]           # one element
```

```
[1] 85
```

There are three common column-extraction operators:

```
students["score"]             # a one-column data frame
```

```
  score
1    90
2    85
3    NA
4    94
5    88
6    91
```

```
students[["score"]]           # the underlying vector
```

```
[1] 90 85 NA 94 88 91
```

```
students$score                # the underlying vector, convenient interactively
```

```
[1] 90 85 NA 94 88 91
```

This distinction affects downstream code. `mean(students$score, na.rm = TRUE)` works on a numeric vector; `students["score"]` remains a data frame.

Use `[[` inside functions when the column name is stored in another object.

```
variable <- "score"
mean(students[[variable]], na.rm = TRUE)
```

```
[1] 89.6
```

2.10.4 Adding and modifying variables

Because each column is a vector, vectorized calculations naturally create new variables.

```
students$passed <- students$score >= 70
students$centered_score <- students$score - mean(students$score, na.rm = TRUE)
students
```

	id	name	program	score	completed	passed	centered_score
1	1	Alice	MS	90	TRUE	TRUE	0.4
2	2	Bob	PhD	85	TRUE	TRUE	-4.6
3	3	Chen	MS	NA	FALSE	NA	NA
4	4	Divya	PhD	94	TRUE	TRUE	4.4
5	5	Elena	MS	88	TRUE	TRUE	-1.6
6	6	Farah	PhD	91	TRUE	TRUE	1.4

Missingness propagates: the student with an unknown score also has an unknown value of passed. This is different from FALSE; we do not know whether that student passed.

2.10.5 Importing rectangular data

Base R provides `read.csv()` and `read.table()` for delimited text files.

```
survey <- read.csv(
  "data/survey.csv",
  na.strings = c("", "NA", ".")
)

measurements <- read.table(
  "data/measurements.txt",
```

```
header = TRUE,  
sep = "\\t"  
)
```

Use project-relative paths. Immediately after importing, check the dimensions, names, types, missingness, and several observations.

```
dim(survey)  
names(survey)  
str(survey)  
head(survey)  
colSums(is.na(survey))
```

i Data import is part of the analysis

Import is not merely clerical. Delimiters, decimal marks, missing-value codes, encodings, date formats, and inferred column types can change the data R sees—and therefore the analysis.

⚠ Do not treat a data frame as a matrix

`as.matrix()` and `apply()` may coerce an entire mixed-type data frame to a character matrix. Perform column-wise operations on the original columns with `lapply()`, `vapply()`, or `dplyr::across()` instead.

2.11 A first look at the tidyverse

The **tidyverse** is a coordinated collection of R packages for data import, transformation, visualization, and programming. Its packages share conventions, but they do not replace base R. In this course, we will learn both because understanding base R explains how objects and functions behave, while tidyverse tools often make data-analysis workflows more readable.

Install the tidyverse once on a computer, then load it in each new R session that uses it.

```
install.packages("tidyverse")
```

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

Loading `tidyverse` attaches several core packages, including `dplyr`, `ggplot2`, `tidyr`, `readr`, `purrr`, `tibble`, `stringr`, and `forcats`. For a lighter dependency, load only the package you need or call a function explicitly, such as `dplyr::filter()`.

2.11.1 Data frames and tibbles

A **tibble** is a modern data frame. It keeps the fundamental data-frame model—equal-length columns that may have different types—but prints compactly and applies stricter, more predictable subsetting rules.

```
students_tbl <- as_tibble(students)
students_tbl
```

```
# A tibble: 6 x 7
   id name  program score completed passed centered_score
  <int> <chr> <chr>   <dbl> <lgl>      <lgl>          <dbl>
1     1 Alice MS        90 TRUE      TRUE          0.400
2     2 Bob  PhD        85 TRUE      TRUE         -4.60
3     3 Chen MS        NA FALSE    NA           NA
4     4 Divya PhD        94 TRUE      TRUE          4.40
5     5 Elena MS        88 TRUE      TRUE         -1.60
6     6 Farah PhD        91 TRUE      TRUE          1.40
```

```
class(students_tbl)
```

```
[1] "tbl_df"      "tbl"        "data.frame"
```

Important differences include:

Operation	Base data frame	Tibble
Printing	May print many rows and columns	Prints a compact preview
<code>x[, "score"]</code>	May simplify to a vector	Remains a one-column tibble
Partial matching with <code>\$</code>	Can occur in some base objects	Warns and returns NULL
Row names	Supported	Discouraged; use an explicit column

Both satisfy `is.data.frame()`, so most functions written for data frames also accept tibbles.

```
is.data.frame(students_tbl)
```

```
[1] TRUE
```

```
students_tbl[, "score"]
```

```
# A tibble: 6 x 1
  score
  <dbl>
1    90
2    85
3    NA
4    94
5    88
6    91
```

```
students_tbl[["score"]]
```

```
[1] 90 85 NA 94 88 91
```

2.11.2 Pipes and data verbs

The native pipe `|>` sends the result on its left to the first argument of the function on its right. It lets us read a transformation in the order it occurs.

```
students_tbl |>
  filter(completed, !is.na(score)) |>
  select(name, program, score) |>
  arrange(desc(score))
```

```
# A tibble: 5 x 3
  name  program score
<chr> <chr>   <dbl>
1 Divya PhD      94
2 Farah PhD      91
3 Alice MS       90
4 Elena MS       88
5 Bob   PhD       85
```

Core `dplyr` verbs have a consistent grammar:

Verb	Purpose
<code>select()</code>	Choose, rename, or reorder columns
<code>filter()</code>	Keep rows satisfying logical conditions
<code>arrange()</code>	Sort rows
<code>mutate()</code>	Create or modify columns
<code>summarise()</code>	Reduce observations to summary values
<code>group_by()</code>	Define groups for subsequent operations

For example, create variables and then compute group-specific summaries:

```
students_tbl |>
  mutate(
    passed = score >= 70,
    score_z = (score - mean(score, na.rm = TRUE)) /
      sd(score, na.rm = TRUE)
  ) |>
  group_by(program) |>
  summarise(
    n = n(),
    n_observed = sum(!is.na(score)),
    mean_score = mean(score, na.rm = TRUE),
    sd_score = sd(score, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 2 x 5
  program      n n_observed mean_score sd_score
  <chr>   <int>   <int>     <dbl>   <dbl>
1 MS         3       2       89     1.41
2 PhD        3       3       90     4.58
```

Grouping changes how later verbs operate. It does not rearrange the rows, and the grouping state can persist. Use `.groups = "drop"` in `summarise()` or `ungroup()` when subsequent operations should apply to the full data.

2.11.3 Operating across several columns

Inside `mutate()` or `summarise()`, `across()` applies one or more functions to a selection of columns.

```
students_tbl |>
  summarise(
    across(
      where(is.numeric),
      list(mean = ~ mean(.x, na.rm = TRUE),
           sd = ~ sd(.x, na.rm = TRUE)),
      .names = "{.col}_{.fn}"
    )
  )
```

```
# A tibble: 1 x 6
  id_mean id_sd score_mean score_sd centered_score_mean centered_score_sd
  <dbl> <dbl>   <dbl>   <dbl>           <dbl>           <dbl>
1   3.5  1.87    89.6    3.36      5.86e-15        3.36
```

`across()` is often the tidyverse counterpart to a column-wise `lapply()` or `vapply()`, with the additional advantage that it works inside grouped data operations.

2.11.4 Base R and tidyverse are complementary

The same task can be expressed in more than one style.

```
# Base R
aggregate(score ~ program, data = students, FUN = mean, na.rm = TRUE)
```

	program	score
1	MS	89
2	PhD	90

```
# tidyverse
students_tbl |>
  group_by(program) |>
  summarise(mean_score = mean(score, na.rm = TRUE), .groups = "drop")
```

```
# A tibble: 2 x 2
  program mean_score
  <chr>      <dbl>
1 MS          89
2 PhD          90
```

Choose a style that makes the operation clear and use it consistently within a pipeline. Regardless of syntax, the underlying questions remain the same: What object enters the function? What object comes out? How are missing values handled? Are types and groups preserved?

2.12 Functions and function calls

A function packages a computation so it can be reused and tested.

```
standardize <- function(x, remove_na = FALSE) {
  center <- mean(x, na.rm = remove_na)
  spread <- sd(x, na.rm = remove_na)
  (x - center) / spread
}

standardize(c(2, 4, 6, 8))
```

```
[1] -1.1618950 -0.3872983  0.3872983  1.1618950
```

A function has three main components:

- **formals:** its arguments;
- **body:** the expression it evaluates;
- **environment:** where it was created.

```
formals(standardize)
```

```
$x
```

```
$remove_na  
[1] FALSE
```

```
body(standardize)
```

```
{  
  center <- mean(x, na.rm = remove_na)  
  spread <- sd(x, na.rm = remove_na)  
  (x - center)/spread  
}
```

```
environment(standardize)
```

```
<environment: R_GlobalEnv>
```

When a function is called, R creates a new temporary environment, binds arguments to values, evaluates the body, and returns the value of the final evaluated expression unless `return()` is used earlier.

2.12.1 Argument matching

Arguments can be matched by exact name or by position. Exact names are clearest in code that others will read.

```
round(x = 3.14159, digits = 3)
```

```
[1] 3.142
```

```
round(3.14159, 3)
```

```
[1] 3.142
```

Avoid partial argument names in durable code; they can be ambiguous and may break if a function changes.

2.12.2 Lazy evaluation

R function arguments are evaluated only when they are needed. This is called **lazy evaluation**.

```
choose_one <- function(x, y) {  
  x  
}  
  
choose_one(10, stop("This argument is never needed"))
```

```
[1] 10
```

Internally, an argument is represented by a *promise*: the expression supplied by the caller, together with the environment in which that expression should be evaluated. Lazy evaluation enables convenient interfaces, but code with side effects can become harder to reason about.

2.13 Environments and lexical scoping

An environment stores name–object bindings and has a parent environment. When R evaluates a name, it looks in the current environment and then follows the chain of parent environments until it finds a binding or reaches the empty environment.

R uses **lexical scoping**: a function looks for nonlocal names in the environment where the function was defined, not the environment from which it was called.

```
make_power <- function(power) {  
  function(x) {  
    x^power  
  }  
}  
  
square <- make_power(2)  
cube <- make_power(3)  
  
square(5)
```

```
[1] 25
```

```
cube(5)
```

```
[1] 125
```

Even after `make_power()` finishes, each returned function retains access to the environment containing its own value of `power`. A function bundled with its defining environment is often called a **closure**.

Local assignment normally changes a binding in the current environment.

```
x <- 100

change_locally <- function() {
  x <- 5
  x
}

change_locally()
```

```
[1] 5
```

```
x
```

```
[1] 100
```

The superassignment operator `<<-` searches parent environments and modifies an existing binding. Because this creates nonlocal side effects, use it sparingly and deliberately.

2.14 Memory: copy-on-modify

R behaves as if objects are copied when modified. In practice, two names may initially share memory, and R copies only when a modification would otherwise affect another binding. This is commonly described as **copy-on-modify**.

```
x <- c(10, 20, 30)
y <- x
y[1] <- 999

x
```

```
[1] 10 20 30
```

```
y
```

```
[1] 999 20 30
```

Changing `y` does not change `x`. This model makes ordinary R code easier to reason about, but repeatedly growing an object can allocate and copy memory many times.

Avoid this pattern for large computations:

```
result <- numeric(0)
for (i in seq_len(100000)) {
  result <- c(result, i^2)
}
```

Preallocate the final size instead:

```
n <- 100000
result <- numeric(n)

for (i in seq_len(n)) {
  result[i] <- i^2
}

head(result)
```

```
[1] 1 4 9 16 25 36
```

`object.size()` gives an approximate size of an R object. `gc()` requests garbage collection and reports memory-use information, although R normally manages garbage collection automatically.

```
object.size(result)
```

```
800048 bytes
```

i Implementation detail: ALTREP

Modern R can represent some vectors compactly and materialize their elements only when necessary. This mechanism, called ALTREP, is one reason an object's conceptual size and its immediate memory cost need not be identical.

2.15 Iteration: vectorization, loops, and the apply family

An iterative computation can often be expressed in several ways. None is universally best. Choose the form that clearly represents the algorithm and returns a predictable object.

```
x <- c(2, 4, 6, 8)

# Vectorized operation
x^2
```

```
[1]  4 16 36 64
```

```
# Explicit, preallocated loop
squares <- numeric(length(x))
for (i in seq_along(x)) {
  squares[i] <- x[i]^2
}
squares
```

```
[1]  4 16 36 64
```

```
# Function applied to each element
vapply(x, function(value) value^2, numeric(1))
```

```
[1]  4 16 36 64
```

Use vectorization when a vectorized function already expresses the operation. Use a loop when the calculation depends on evolving state or complex indexing. Use an apply-family function when the same function is independently applied to many pieces of an object.

! The apply family is not magic vectorization

`apply()` and `lapply()` usually perform iteration internally. They are not automatically faster than a well-written, preallocated `for` loop. Their main benefit is expressing “apply this function to each piece” concisely.

2.15.1 `apply()`: rows or columns of a matrix

`apply(X, MARGIN, FUN, ...)` applies `FUN` over dimensions of an array or matrix:

- `MARGIN = 1` operates on rows;
- `MARGIN = 2` operates on columns;
- `MARGIN = c(1, 2)` operates on individual cells.

```
X <- matrix(1:12, nrow = 3, byrow = TRUE)
colnames(X) <- c("x1", "x2", "x3", "x4")
X
```

```
      x1 x2 x3 x4
[1,]  1  2  3  4
[2,]  5  6  7  8
[3,]  9 10 11 12
```

```
apply(X, MARGIN = 1, FUN = mean) # row means
```

```
[1]  2.5  6.5 10.5
```

```
apply(X, MARGIN = 2, FUN = sd)    # column standard deviations
```

```
x1 x2 x3 x4
4  4  4  4
```

Additional arguments are passed to `FUN` through `....`

```
X_missing <- X
X_missing[2, 3] <- NA
apply(X_missing, 2, mean, na.rm = TRUE)
```

```
x1 x2 x3 x4
  5  6  7  8
```

For common matrix summaries, specialized functions such as `rowMeans()`, `colMeans()`, `rowSums()`, and `colSums()` are clearer and generally faster.

```
rowMeans(X)
```

```
[1]  2.5  6.5 10.5
```

```
colSums(X)
```

```
x1 x2 x3 x4
15 18 21 24
```

2.15.2 Why `apply()` can be dangerous for data frames

`apply()` first converts a data frame to a matrix. If every column is numeric, this can be acceptable. If any column is character, the matrix may become entirely character, making numerical summaries fail or behave unexpectedly.

```
mixed_df <- data.frame(
  id = c("A", "B", "C"),
  x = c(10, 20, 30),
  y = c(2, 4, 6)
)
```

```
as.matrix(mixed_df)
```

```
      id  x  y
[1,] "A" 10 2
[2,] "B" 20 4
[3,] "C" 30 6
```

```
typeof(as.matrix(mixed_df))
```

```
[1] "character"
```

Select the numeric columns explicitly before a matrix operation, or use a column-wise tool that preserves each column's type.

```
numeric_columns <- vapply(mixed_df, is.numeric, logical(1))
vapply(mixed_df[numeric_columns], mean, numeric(1))
```

```
  x  y
20  4
```

2.15.3 lapply(): apply a function and return a list

`lapply(X, FUN, ...)` works naturally with lists and data frames because a data frame is a list of columns. It always returns a list of the same length as `X`.

```
lapply(students, class)
```

```
$id
[1] "integer"

$name
[1] "character"

$program
[1] "character"

$score
[1] "numeric"

$completed
[1] "logical"

$passed
[1] "logical"

$centered_score
[1] "numeric"
```

```
lapply(students[c("score", "centered_score")], mean, na.rm = TRUE)
```

```
$score  
[1] 89.6
```

```
$centered_score  
[1] 5.861978e-15
```

The stable list return type makes `lapply()` predictable, even when the individual results have different lengths or types.

```
lapply(students, function(column) {  
  c(type = typeof(column), length = length(column))  
})
```

```
$id  
      type      length  
"integer"      "6"
```

```
$name  
      type      length  
"character"      "6"
```

```
$program  
      type      length  
"character"      "6"
```

```
$score  
      type      length  
"double"      "6"
```

```
$completed  
      type      length  
"logical"      "6"
```

```
$passed  
      type      length  
"logical"      "6"
```

```
$centered_score  
      type      length  
"double"      "6"
```

2.15.4 sapply() and vapply(): simplified output

sapply() calls lapply() and then attempts to simplify the result to a vector, matrix, or array.

```
sapply(students, class)
```

id	name	program	score	completed
"integer"	"character"	"character"	"numeric"	"logical"
passed	centered_score			
"logical"	"numeric"			

```
sapply(students[c("score", "centered_score")], mean, na.rm = TRUE)
```

score	centered_score
8.960000e+01	5.861978e-15

The convenience comes with a risk: the return type depends on the data. vapply() requires a template describing one result and therefore fails early if the function returns an unexpected type or length.

```
vapply(students, typeof, character(1))
```

id	name	program	score	completed
"integer"	"character"	"character"	"double"	"logical"
passed	centered_score			
"logical"	"double"			

```
vapply(students[c("score", "centered_score")],  
       mean,  
       numeric(1),  
       na.rm = TRUE)
```

score	centered_score
8.960000e+01	5.861978e-15

Prefer vapply() when writing reusable functions or production analyses with a known output type.

2.15.5 `tapply()`: grouped operations on a vector

`tapply(X, INDEX, FUN, ...)` splits a vector into groups and applies a function to each group.

```
with(students, tapply(score, program, mean, na.rm = TRUE))
```

```
MS PhD  
89 90
```

For multiple grouping variables, pass a list of group vectors.

```
with(students, tapply(score,  
                        list(program = program, completed = completed),  
                        mean,  
                        na.rm = TRUE))
```

```
      completed  
program FALSE TRUE  
  MS      NaN   89  
  PhD      NA   90
```

`aggregate()` extends the split-apply-combine idea to data frames, while `dplyr::group_by()` followed by `summarise()` expresses it through the tidyverse grammar.

2.15.6 `Map()` and `mapply()`: parallel iteration

`Map()` applies a function to corresponding elements of two or more inputs and returns a list. `mapply()` attempts to simplify the result.

```
Map(function(center, spread) rnorm(3, center, spread),  
      center = c(0, 10),  
      spread = c(1, 2))
```

```
[[1]]  
[1] -0.2662771  0.3798825  0.7719476
```

```
[[2]]  
[1] 9.001981 9.896442 9.821075
```

The tidyverse package `purrr` provides related type-stable tools: `map()` returns a list, while `map_dbl()`, `map_int()`, `map_chr()`, and `map_lgl()` require a particular atomic return type.

```
map_chr(students, ~ class(.x)[1])
```

```

      id      name      program      score      completed
"integer" "character" "character" "numeric" "logical"
  passed centered_score
"logical"      "numeric"
```

```
map_dbl(students[c("score", "centered_score")],
  ~ mean(.x, na.rm = TRUE))
```

```

      score centered_score
8.960000e+01  5.861978e-15
```

2.15.7 Choosing an iteration tool

Goal	Recommended starting point
Element-wise arithmetic	A vectorized function or operator
Standard row/column sums or means	<code>rowSums()</code> , <code>colSums()</code> , <code>rowMeans()</code> , <code>colMeans()</code>
General operation on matrix rows/columns	<code>apply()</code>
Operation on list or data-frame columns; list output	<code>lapply()</code> or <code>purrr::map()</code>
Known scalar output for each element	<code>vapply()</code> or a typed <code>purrr::map_*()</code>
One vector summarized by groups	<code>tapply()</code>
Data-frame columns inside a tidy pipeline	<code>dplyr::across()</code>
Computation with evolving state or complex indexing	A preallocated <code>for</code> loop

2.16 Numerical computation is finite-precision computation

Most real numbers cannot be stored exactly in binary floating-point arithmetic.

```
0.1 + 0.2
```

```
[1] 0.3
```

```
(0.1 + 0.2) == 0.3
```

```
[1] FALSE
```

```
all.equal(0.1 + 0.2, 0.3)
```

```
[1] TRUE
```

Use tolerance-based comparisons, such as `all.equal()`, for computed floating-point values. Numerical algorithms must also consider overflow, underflow, cancellation, conditioning, and convergence—not only algebraic correctness.

A simple example of overflow avoidance is computing a log-likelihood with `dbinom(..., log = TRUE)` rather than taking the logarithm after a tiny probability has already been rounded to zero.

```
log(dbinom(500, size = 1000, prob = 0.01))
```

```
[1] -Inf
```

```
dbinom(500, size = 1000, prob = 0.01, log = TRUE)
```

```
[1] -1618.143
```

2.17 Randomness and reproducibility

Pseudo-random numbers are generated deterministically from an internal state. Setting the seed makes a simulation reproducible.

```
set.seed(8670)  
rnorm(5)
```

```
[1] -0.4916281  1.1467524  1.5963118 -0.6655463  0.5516107
```

```
set.seed(8670)  
rnorm(5)
```

```
[1] -0.4916281  1.1467524  1.5963118 -0.6655463  0.5516107
```

Set a seed at a meaningful boundary—usually once near the start of a simulation—not repeatedly inside the loop being studied. Record the code, data provenance, package versions, and R session information needed to reproduce an analysis.

```
sessionInfo()
```

```
R version 4.6.1 (2026-06-24)
```

```
Platform: aarch64-apple-darwin23
```

```
Running under: macOS Tahoe 26.6.2
```

```
Matrix products: default
```

```
BLAS:   /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
```

```
LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib; LAPACK
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: America/New_York
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
```

```
[1] lubridate_1.9.5 forcats_1.0.1  stringr_1.6.0  dplyr_1.2.1  
[5] purrr_1.2.2     readr_2.2.0    tidyr_1.3.2    tibble_3.3.1  
[9] ggplot2_4.0.3   tidyverse_2.0.0
```

```
loaded via a namespace (and not attached):
```

```
[1] gtable_0.3.6      jsonlite_2.0.0    compiler_4.6.1    tidyselect_1.2.1  
[5] scales_1.4.0      yaml_2.3.12       fastmap_1.2.0     R6_2.6.1  
[9] generics_0.1.4    knitr_1.51        pillar_1.11.1     RColorBrewer_1.1-  
3  
[13] tzdb_0.5.0        rlang_1.3.0       utf8_1.2.6        stringi_1.8.9  
[17] xfun_0.60         S7_0.2.2          otel_0.2.0        timechange_0.4.0  
[21] cli_3.6.6         withr_3.0.3       magrittr_2.0.5    digest_0.6.39  
[25] grid_4.6.1        rstudioapi_0.19.0 hms_1.1.4         lifecycle_1.0.5  
[29] vctrs_0.7.3       evaluate_1.0.5    glue_1.8.1        farver_2.1.2  
[33] codetools_0.2-20  rmarkdown_2.31    tools_4.6.1       pkgconfig_2.0.3  
[37] htmltools_0.5.9
```

2.18 Packages, help, and error messages

Packages extend R with functions, data, and documentation.

Most general-purpose packages are distributed through the Comprehensive R Archive Network (CRAN). Bioconductor distributes packages for bioinformatics and high-throughput biological data, and development versions of some packages are hosted on GitHub.

```
# CRAN
install.packages("packageName")

# Bioconductor: install BiocManager from CRAN first
BiocManager::install("packageName")

# GitHub: install the remotes package from CRAN first
remotes::install_github("username/repository")

# Load an installed package in the current session
library(packageName)

# Or call one function without attaching the package
packageName::function_name()
```

Installation changes the R library on the computer and normally happens once. `library()` makes an installed package available in the current session and must be repeated in each new session that uses it.

Useful ways to investigate an unfamiliar function include:

```
?mean
help("mean")
example(mean)
args(mean)
methods(mean)
```

When an error occurs:

1. Read the entire message.
2. Reduce the problem to the smallest input that still fails.
3. Inspect object structures and dimensions with `str()`, `typeof()`, `class()`, and `dim()`.
4. Check the most recent calls with `traceback()` after an error.
5. Verify assumptions explicitly with `stopifnot()`.

```
safe_mean <- function(x) {
  stopifnot(is.numeric(x), length(x) > 0)
  mean(x)
}
```

```
safe_mean(c(2, 4, 6))
```

```
[1] 4
```

2.19 A computational workflow for this course

A reliable statistical computation separates the following concerns:

1. **Inputs:** raw data, constants, configuration, and seeds.
2. **Transformation:** cleaning and construction of analysis variables.
3. **Computation:** algorithms, models, optimization, or simulation.
4. **Validation:** diagnostics, invariants, tests, and sensitivity checks.
5. **Communication:** tables, graphics, interpretation, and limitations.

Prefer scripts and Quarto documents over a sequence of unrecorded console commands. A clean R session should be able to run the analysis from beginning to end. Do not rely on objects left over from earlier experimentation.

```
# A small, reproducible computation
set.seed(8670)

n <- 200
x <- rnorm(n)
y <- 1 + 2 * x + rnorm(n, sd = 0.75)
simulated_data <- data.frame(x = x, y = y)

fit <- lm(y ~ x, data = simulated_data)
coef(fit)
```

```
(Intercept)          x
  1.020794    1.991843
```

The code states where the data came from, fixes the random seed, stores the data in an explicit object, and passes that object to the model.

2.20 In-class exercises

Predict the value and type of each expression. Then run it and explain the result.

```
c(TRUE, 2L, 3.5)
c(1, 2, 3, 4) + c(10, 20)
c(1, 2, NA) > 1
TRUE | NA
```

Using the vector below, write expressions that return:

1. all nonmissing values;
2. values greater than or equal to 10;
3. the positions of values greater than or equal to 10;
4. the mean of the observed values.

```
x <- c(4, NA, 10, 15, 3, NA, 21)
```

Without running the code, determine the result of `f(5)`. Identify where each occurrence of `a` is found.

```
a <- 10

f <- function(x) {
  a <- 2
  g <- function(y) a * y
  g(x)
}

f(5)
```

Write a function that takes a positive integer `n` and returns a vector whose i th element is

$$\frac{(-1)^{i+1}}{i}.$$

Preallocate the result and use `seq_len(n)`.

Estimate the probability that the mean of 20 independent standard normal variables exceeds 0.5. Use at least 10,000 Monte Carlo replications and set a seed. Then compare your estimate with the exact probability computed from the sampling distribution of the mean.

Using `students`, complete each task once with base R and once with `dplyr`:

1. retain students with observed scores;
2. select `name`, `program`, and `score`;
3. create a variable equal to `score - 90`;
4. sort from highest to lowest score;
5. compute the number of observed scores and mean score within each program.

Before running the code, predict whether each result is a vector, data frame, tibble, or grouped tibble.

For each task, choose among a vectorized function, `apply()`, `lapply()`, `vapply()`, `tapply()`, `dplyr::across()`, or a loop. Justify the choice, then write the code.

1. Compute the mean of every numeric column in `students`.
2. Compute the row sums of a 10,000-by-50 numerical matrix.
3. Record the class of every column in a data frame.
4. Compute the mean score within each program.
5. Fit the same model to each of 20 independently generated data sets and retain the complete fitted-model objects.

2.20.1 Exercise 8: Diagnose coercion

Predict the type and contents of `result`. Explain why `apply()` is inappropriate here, and rewrite the computation using `vapply()` or `dplyr::across()`.

```
dat <- data.frame(
  subject = c("A", "B", "C"),
  pre = c(10, 12, 9),
  post = c(14, 13, 15)
)

result <- apply(dat, 2, mean)
```

2.21 Takeaways

- R evaluates expressions in environments and binds names to objects.
- Atomic vectors are the foundation of R computation; vectorization, coercion, and recycling follow from this design.
- Lists, matrices, factors, and data frames build on vectors and attributes.
- A data frame is a named list of equal-length columns; a tibble is a stricter, modern data frame.

- `dplyr` expresses row, column, transformation, and grouped-summary operations as a readable pipeline.
- Functions are objects with arguments, a body, and a defining environment; R uses lexical scoping and lazy evaluation.
- The `apply` family expresses repeated operations, but the input structure and promised output type determine which member is appropriate.
- `apply()` is designed for arrays and matrices; using it on a mixed-type data frame can trigger unwanted coercion.
- R's copy-on-modify behavior favors clear semantics, but allocation patterns still matter for performance.
- Statistical computing requires attention to missing values, floating-point arithmetic, random-number state, and reproducibility.
- Understanding R's computational model makes code easier to predict, debug, optimize, and trust.

2.22 Optional challenge

Explain why the following two functions may have very different memory behavior even though they return the same values. Design a timing experiment for increasing values of `n`.

```
grow <- function(n) {
  out <- numeric(0)
  for (i in seq_len(n)) {
    out <- c(out, i^2)
  }
  out
}

preallocate <- function(n) {
  out <- numeric(n)
  for (i in seq_len(n)) {
    out[i] <- i^2
  }
  out
}
```

This appendix is adapted from [Why R?](#).